



St. Petersburg State University
Software Engineering Chair

Atomic Unit in Vortex GPU

Shamil Shaehmitov, SPBU SE

Saint Petersburg
2026

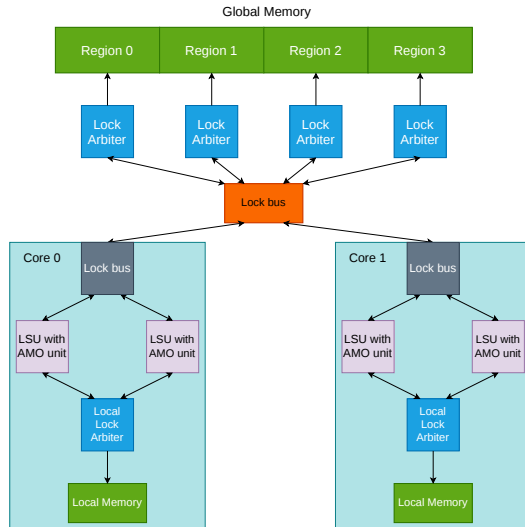
Introduction

- Spla: sparse-linear-algebra-based graph analysis library that uses OpenCL
- Our goal: to run Spla on Vortex and conduct experiments
- Linear algebra kernels heavily rely on atomic operations
 - ✓ Support for atomics has been added in PoCL
 - ☹ In the RTL design, support for extension RISC-V A was missing
- We have implemented atomic operation support in RTL
 - ! Designed to enable experiments on FPGA, not to achieve optimal performance
 - ⚙ The implementation is currently under active development

Memory Lock Arbiters

- The memory was divided into regions, each of which was assigned an arbiter
- The arbiters for global memory are located at the topmost level
- When one of the LSUs needs to perform an atomic operation in a region, the arbiter grants permission in a round-robin order
- The same arbiters are implemented for local memory if the core has more than one LSU
- The number of regions is configurable

Memory Lock Arbiters



Atomic Unit

- Placed in the LSU
- The AMO unit is connected to the arbiters via a bus that allows the unit to request a lock and the arbiters to grant it
- It is a finite state machine with 9 states
- States
 - ▶ IDLE: waiting for a request
 - ▶ LOCK: waiting for the lock to be granted
 - ▶ DRAIN: waiting for the pipeline to be drained
 - ▶ READREQ: sending a read request
 - ▶ READRSP: waiting for a read response
 - ▶ WRITEREQ: sending a write request
 - ▶ WRITERSP: waiting for a write response
 - ▶ FENCEREQ: sending a dummy read request before lock release
 - ▶ FENCERSP: waiting for a dummy read response

Optimization 1[WIP]

- Memory banks are processed sequentially
- This makes it possible to complete all operations with fewer lock requests
- It also requires entering the FENCE state less often

Optimization 2[WIP]

- Skipping read and write operations for consecutive operations to the same address
- The read occurs during the first operation
- The write occurs during the last operation

LR/SC instructions

- LR/SC instructions are implemented using the same atomic unit
- The LR instruction is implemented as a read operation with a lock request
- The SC instruction is implemented as a write operation with a lock request
- Two register arrays of size $WARPS_PER_LSU * NUM_THREADS$ are used to track the reservation: reservation address and data

Conclution

- Our implementation
 - ▶ Atomic Unit: https://github.com/talubik/vortex/tree/amo_unit_rtl
 - ▶ Atomic operations in PoCL: <https://github.com/vortexgpgpu/pocl/commit/70db40e299b7a8e4a7e3897339621030f626bb48>
- Future work
 - ▶ Implement basic improvements
 - ▶ Run Spla in RTL simulation and on FPGA
 - ▶ Compare our atomics with [baseline](#)
 - ▶ What about “Atomic Cache: Enabling Efficient Fine-Grained Synchronization with Relaxed Memory Consistency on GPGPUs through In-Cache Atomic Operations”?

Our Pull Requests with various fixes for Vortex:

- CopyBuff operation support: <https://github.com/vortexgpgpu/vortex/pull/310>
- VX Spawn Fix: <https://github.com/vortexgpgpu/vortex/pull/321>